Nombre de la formación

Análisis y desarrollo de software

Ficha:

3336220

Aprendiz Servicio nacional de aprendizaje

SENA

Actividad de aprendizaje:

Informe técnico de plan de trabajo para construcción de software.GA7-220501096-AA1-EV01.

Competencia:

Análisis y desarrollo de software

Trimestre:

T_3

Instructor:
FABIAN ALBERTO ALBORNOZ AYALA

Fecha:

29 de octubre del 2026

Santiago de Cali

IDENTIFICACIÓN DE LA ACTIVIDAD DE APRENDIZAJE

Denominación del Programa de Formación:
TECNÓLOGO EN ANÁLISIS Y DESARROLLO DE FOTWARE VIRTUAL

Código del Programa de Formación:
3336220

Nombre del Proyecto:
Informe técnico de plan de trabajo para construcción de software.GA7-220501096-AA1-EV01.

Fase del Proyecto:
Ejecución

Actividad de Proyecto:
Informe técnico de plan de trabajo para construcción de software.GA7-220501096-AA1-EV01.

Competencia
Diseñar la solución de software de acuerdo con procedimientos y requisitos técnicos.

Resultados de Aprendizaje Alcanzar:
Crear un modelo de base de datos, con base a los requerimientos del cliente.

Duración: 96 horas Horas.

Entregable:
Formato de entrega de aprendizaje proyecto

Nombre Aprendices
Gustavo Henry Gómez Gutiérrez

Fecha:
29 de octubre del 2026

TABLA DE CONTENIDO

INTRODUCCIÓN 5
JUSTIFICACIÓN 6
OBJETIVO GENERAL 7
OBJETIVOS ESPECIFICOS.....7
DESARROLLO DE LA ACTIVIDAD.....8
CONCLUSIÓN 9
REFERENCIA 11

INTRODUCCIÓN

El desarrollo de sistemas de software modernos exige una estructuración metódica que trascienda la simple escritura de código. La ingeniería de software contemporánea demanda la integración armónica entre un paradigma de diseño sólido y una infraestructura de herramientas que soporte la colaboración simultánea, el control de cambios rigoroso y la entrega de valor constante. Este informe técnico formaliza el plan de trabajo para la fase de construcción de software, fundamentándose estrictamente en las directrices de los componentes formativos "Aplicación del paradigma orientado a objetos en el lenguaje de programación" e "Integración continua" (SENA, 2026). A través de esta especificación se definen los estándares de codificación orientada a objetos en Java, la organización del dominio mediante modelado de entidades y las herramientas de gestión de código fuente que regirán el flujo de trabajo del equipo de desarrollo.

JUSTIFICACIÓN

La adopción de un plan de trabajo técnico estructurado responde a la necesidad crítica de mitigar riesgos operativos durante la fase de construcción del software, tales como la duplicidad de esfuerzos, la pérdida de trazabilidad en los cambios de código, los conflictos en la integración de funcionalidades y la degradación de la calidad arquitectónica. La implementación del paradigma de Programación Orientada a Objetos (POO) proporciona la abstracción, encapsulación y modularidad requeridas para representar adecuadamente las reglas de negocio (SENA, 2026). Por su parte, la incorporación explicita de tecnologías de versiona miento y la automatización de la integración continua garantizan la estabilidad del producto final en entornos multiusuario, optimizando los tiempos de compilación, verificación y despliegue dentro del ciclo de desarrollo de software.

OBJETIVO GENERAL

Elaborar el informe técnico del plan de trabajo para la construcción de software, especificando la selección de herramientas de versionamiento, la arquitectura orientada a objetos y las estrategias de integración continua, para garantizar un desarrollo ordenado, escalable y alineado con las buenas prácticas de la industria.

OBJETIVOS ESPECIFICOS

- Definir la estructura arquitectónica del software basada en los principios del paradigma orientado a objetos en la plataforma Java.
- Seleccionar las herramientas tecnológicas para el control de versiones local y remoto que soporten el desarrollo colaborativo del proyecto.
- Establecer el flujo de trabajo de ramificación (branching strategy) para asegurar la trazabilidad e integridad de la línea base de código.
- Formular la estrategia de automatización e Integración Continua (CI) para la verificación temprana de errores durante la construcción.

DESARROLLO DE LA ACTIVIDAD

Para la construcción de las capas lógicas del sistema, se adopta el lenguaje de programación Java ejecutado sobre la Máquina Virtual de Java (JVM), garantizando independencia de la plataforma, gestión automática de memoria mediante el recolector de basura (Garbage Collector) y soporte nativo para la ejecución multihilo (SENA, 2024a). La arquitectura del modelo de dominio se estructura bajo los pilares fundamentales de la POO:

- **Encapsulación y Ocultamiento de Datos:** Definición de atributos en estado privado (private) con acceso controlado exclusivamente mediante métodos accesores (get) y modificadores (set).
- **Herencia y Extensibilidad:** Reutilización de comportamiento y atributos mediante la cláusula extends, estableciendo jerarquías entre superclases y subclases especializadas.
- **Polimorfismo y Sobrecarga:** Implementación de múltiples formas para métodos homónimos diferenciados por la firma de sus parámetros (overload) y ligadura dinámica en tiempo de ejecución.

Elemento del Paradigma	Sintaxis / Convención en Java	Propósito en el Plan de Construcción
Clases y Plantillas	`public class NombreClase {}`	Modelado de entidades del negocio en notación camelCase comenzando en mayúscula.
Encapsulamiento	`private double precio;`	Protección del estado interno contra modificaciones no autorizadas fuera de la clase.
Constructores	`public Clase(int id, String nom)`	Inicialización garantizada de objetos en memoria con estados válidos iniciales.
Métodos Get / Set	`public String getNombre()`	Mecanismos públicos regulados para la lectura y escritura de atributos encapsulados.
Relaciones y Herencia	`public class SubClase extends Base`	Modelado de relaciones de asociación, agregación, composición y herencia jerárquica.

El entorno de desarrollo integrado (IDE) seleccionado para la fase de construcción comprende soluciones estándar como Apache NetBeans, Eclipse o IntelliJ IDEA, los cuales brindan soporte nativo para el análisis sintáctico de Java, refactorización avanzada y compilación directa a bytecode (SENA, 2024a).

SELECCIÓN DE HERRAMIENTAS DE VERSIONAMIENTO E INTEGRACIÓN CONTINUA

El control de cambios y la integración de código se han diseñado bajo un esquema distribuido que elimina los puntos de fallo único y fomenta el trabajo en paralelo. De acuerdo con el componente formativo de Integración Continua (SENA, 2024b), la suite tecnológica seleccionada abarca cuatro componentes primarios:

1. Control de Versiones: Git

Sistema de control de versiones distribuido (DVCS) utilizado a nivel local. Permite registrar el historial de cambios mediante commits atómicos, gestionar ramas de trabajo independientes y realizar fusiones (merges) con trazabilidad completa **(Chacon & Straub, 2014)**.

2. Repositorio Remoto: GitHub / GitLab

Plataforma basada en la nube para alojar los repositorios de Git. Proporciona control de acceso, gestión de revisiones de código mediante Pull Requests (PR) / Merge Requests (MR), seguimiento de requerimientos y auditoría de cambios.

3. Gestor de Construcción: Apache Maven / Gradle

Herramienta para la gestión de dependencias y automatización del ciclo de vida del proyecto Java. Estandariza la estructura de directorios, resuelve librerías externas de forma automática y ejecuta la compilación del bytecode.

4. Servidor CI: GitHub Actions / Jenkins

Motor de automatización encargado de ejecutar pipelines continuos ante cada evento de `push` o `pull_request`, compilando el código, corriendo pruebas unitarias y verificando la calidad antes de liberar la versión.

ESTRATEGIA DE RAMIFICACIÓN (GIT FLOW WORKFLOW)

- Se adopta una adaptación de la estrategia Git Flow para la organización del trabajo en equipo
- **main / master:** Contiene exclusivamente código en estado de producción, estable y verificado.
- **develop:** Rama de integración continua donde se consolidan las funcionalidades probadas.
- **feature/*:** Ramas temporales creadas a partir de develop para el desarrollo de requerimientos específicos.
- **hotfix/*:** Ramas de emergencia para la corrección inmediata de fallos en producción.

CONCLUSIÓN

El desarrollo de este informe técnico formaliza la hoja de ruta metodológica y tecnológica para la fase de construcción de software, concluyendo los siguientes aspectos clave:

1. Sólida Arquitectura de Software: La aplicación rigurosa de los principios del paradigma orientado a objetos en Java (encapsulamiento, herencia y polimorfismo) garantiza la creación de componentes modulares, fáciles de mantener y preparados para la evolución de los requerimientos del sistema.
2. Garantía de Trazabilidad: La selección de Git como sistema de versionamiento local y GitHub/GitLab como plataforma remota asegura la trazabilidad absoluta de cada modificación en la línea base de código, previniendo sobreescrituras y pérdidas de información.
3. Optimización del Trabajo Colaborativo: La estandarización de la estrategia de ramificación (Git Flow) define reglas claras de interacción entre desarrolladores, aislando las ramas de producción de aquellas en fase de desarrollo activo.
4. Cultura de Calidad Automática: La definición de un pipeline de Integración Continua (CI) minimiza la detección tardía de inconsistencias en la compilación y errores lógicos, reduciendo significativamente los costos de mantenimiento en el ciclo de vida de desarrollo.

REFERENCIA

- *SENA. (2024a). Aplicación del paradigma orientado a objetos en el lenguaje de programación [Componente Formativo]. Servicio Nacional de Aprendizaje (SENA). Regional Antioquia.*
- *SENA. (2024b). Integración continua [Componente Formativo]. Servicio Nacional de Aprendizaje (SENA). Chacon, S., & Straub, B. (2014). Pro Git (2.ª ed.). Apress.*
- *Archivo General de la Nación. (2022). Guía para la formulación de un esquema de metadatos para la gestión de documentos. Gobierno de Colombia.*
- *Instituto Colombiano de Normas Técnicas y Certificación (ICONTEC). (2022). Normas técnicas de la información y seguridad de la familia ISO 27000. ICONTEC.*